MENTOR PRESENTATION — ESSENTIAL GUIDE

What You Need to Know

No syntax. No memorization. Just the concepts, the words to say, and the confidence to say them.

> **How to use this guide:** Read it the night before. Focus on the green *"Say this"* boxes — those are your words for the meeting. You don't need to memorize code. You need to understand *what the app does and why you made each choice*.

1 The Big Picture — What Is Your App?

If your mentor asks: "Tell me about your project"

You need to explain the **problem**, the **solution**, and the **users** — in under 30 seconds.

> **Say this:** *"I built MedFind — a prescription management web app. The problem is that prescriptions today are paper-based: doctors write them, patients carry them, pharmacists manually look up stock. My app makes this digital. A doctor writes a prescription online, the patient sees it instantly, and the pharmacy checks their stock in real time. There are four types of users: doctors, patients, pharmacists, and an admin."*

Doctor

→ Write prescriptions
→ See connected patients
→ Browse pharmacies

Patient

→ View their prescriptions
→ Daily dose schedule
→ Connect to a doctor

Pharmacist

→ Manage medicine inventory
→ Fulfill prescription orders
→ Track low stock

Admin

→ Manage all users
→ Verify doctor accounts
→ Full system access

2 The Tech Stack — What You Used & Why

Key rule: For every technology, know two things — *what it does* and *why you chose it*. That's all your mentor needs.

Technology	One-line explanation	Why you chose it
React	Builds the visual interface (buttons, pages, forms)	Industry standard, component-based, huge community
Firebase Auth	Handles login and signup securely	No need to build a backend — Firebase handles it
Firebase Firestore	Cloud database that stores all app data	Real-time, no backend server needed, free tier
React Router	Controls which page shows for which URL	Standard routing library for React apps
Tailwind CSS	Styles everything (colours, layout, spacing)	Faster than writing regular CSS, consistent design
Radix UI / shadcn	Pre-built UI components (dialogs, dropdowns, cards)	Professional quality, saves hours of UI work
Vite	Runs the app in development (the dev server)	Very fast, modern, easy setup

If asked: "Why Firebase instead of your own backend?"

Say this: *"As a frontend developer, building my own backend — a server, an API, a database — would have doubled the project scope. Firebase gives me authentication and a database out of the box. It let me focus on the actual features: prescriptions, roles, inventory. It's a real-world tool used by thousands of companies, so it's a solid choice."*

3 The 3 Most Important Concepts to Understand

These are the concepts your mentor will almost certainly ask about. Understand *what they mean* — not the code.

Concept A — Role-Based Access Control (RBAC)

What it means: Different users see different things based on their role. A doctor can write prescriptions. A patient cannot. A pharmacist can manage inventory. Others cannot. This is enforced at two levels:

1. Route level — A component called *RequireRole* wraps every page. Before showing the page, it checks the user's role. Wrong role = redirected away.

2. Database level — Firebase security rules on Firestore prevent users from reading or writing data they don't own.

> **Say this:** *"I built a role guard component that wraps every protected route. When a user navigates to a page, the guard checks their role from the global auth context. If the role doesn't match what the page allows, they're sent to an Unauthorized page. This works even if someone manually types the URL — they still can't get in."*

Concept B — React Context (Global State)

What it means: When a user logs in, the app needs to know: *Who is this person? What is their role?* This information needs to be available everywhere in the app — the sidebar, the dashboard, the route guard.

React Context is like a **shared notice board** the whole app can read. After login, the user's info and role are pinned to that board. Every component reads from it.

> **Say this:** *"I use React Context to store the logged-in user's profile and role. After Firebase authenticates the user, I fetch their profile from Firestore and store it in context. Then every*

> *component in the app — the sidebar, the role guard, the dashboard — can access this information without me passing it through props."*

Concept C — Lazy Loading

What it means: Instead of downloading the entire app when the page first loads, only the minimum necessary code is loaded. The doctor's prescription pages, the patient's schedule, the pharmacy inventory — these are separate "chunks" that only download when the user navigates to them.

This makes the **initial load faster**, especially important for users on slower connections.

> **Say this:** *"I used React's lazy() and Suspense to split the app into chunks. Each role's pages are only downloaded when someone visits them. If a patient logs in, they never download the doctor's or pharmacy's code. This keeps the initial load fast."*

4 The Core Feature — Prescription Workflow

This is the heart of the app. Know this flow by heart — your mentor will ask about it.

Patient connects to doctor → Doctor writes prescription → Saved to Firestore → Patient views prescription → Pharmacist fulfills order

Each step explained simply

Step 1 — Connection: A doctor can only prescribe for patients they are connected to. The patient sends a request, the doctor accepts. This prevents random prescriptions.

Step 2 — The form: The doctor picks the patient, types the diagnosis, then adds medicines. Each medicine has a name, dosage (e.g. 500mg), how often (e.g. twice daily), and for how long (e.g. 30 days). They can add multiple medicines in one prescription.

Step 3 — Saving: When submitted, a prescription record is saved to Firestore. Each medicine gets its own item record linked to that prescription. The expiry date is calculated automatically.

Step 4 — Patient view: The patient can see all their prescriptions and a daily dose schedule based on what medicines they are taking.

Step 5 — Pharmacy: Pharmacists see incoming orders, check their stock, and mark orders as filled.

If asked: "What validation does the prescription form have?"

> **Say this:** *"The form checks several things before saving: the doctor must be verified by the admin, a patient must be selected, and at least one medicine must be added with a dosage and frequency. If any of these fail, a clear error message shows and nothing is saved."*

5 How to Handle Tough Questions

Remember: Your mentor is not trying to trick you. They ask hard questions to see *how you think*, not to see if you've memorised everything. Honest, thoughtful answers are always better than guessing.

"What would you improve if you had more time?"

Say this: *"I'd add real-time notifications — so the patient gets an alert the moment a prescription is created. I'd also add PDF export for prescriptions, proper automated testing, and eventually a mobile version. The current code structure makes these additions straightforward since each role is isolated in its own folder."*

"What was the hardest part to build?"

Say this: *"The role-based routing was the trickiest part. I needed every page to be protected, but the user's role only becomes available after an async Firebase call. I had to make sure the loading state was handled correctly — so the app doesn't flash an Unauthorized page before the profile is even loaded."*

"What did you learn from this project?"

Say this: *"I learned how to think architecturally — how to organise a real app so it stays manageable as it grows. I also learned how authentication and authorization are different things: authentication is 'who are you?', authorization is 'what are you allowed to do?' Both are handled differently in this project."*

"I don't know the answer to this" — what to say

Say this: *"That's a great question — I haven't explored that part yet. Can I note it down and come back to you after I've researched it?"*

Authentication vs Authorization

Authentication = "Who are you?" (login with email/password)
Authorization = "What can you do?" (role guard checks your role)

Frontend vs Backend

Frontend = what the user sees (React, Tailwind, your code)
Backend = where data is stored (Firebase — you didn't build this yourself)

Component vs Page

Component = a reusable piece of UI (a button, a card, a form)
Page = what shows for a URL, usually made of multiple components

State vs Props

State = data a component manages itself (e.g. loading, error)
Props = data passed into a component from outside (e.g. the list of medicines)

6 The Demo Plan — 12 Minutes, Step by Step

> **Before the meeting:** Run `npm run dev`. Create test accounts in advance. Have a sample prescription already saved so there is real data to show.

1. **Login page (1 min)**
   Show the sign-in form. Say: "Firebase handles authentication here — I don't store passwords, Firebase does that securely." Log in as a doctor.

2. **Doctor dashboard (2 min)**
   Show the welcome screen and sidebar. Say: "The sidebar only shows pages this role can access. A patient logging in would see a completely different menu."

3. **New Prescription page (3 min) — your star feature**
   Walk through the form slowly. Select a patient. Add a medicine with dosage and frequency. Submit. Show the success redirect. Say: "The prescription is now in Firestore and the patient can see it immediately."

4. **Switch to patient account (2 min)**
   Log in as a patient. Open the prescriptions list — the prescription you just wrote is there. Then show the dose schedule. Say: "Same data, different perspective."

5. **Switch to pharmacy account (2 min)**
   Show the inventory dashboard with the stats cards. Open the orders list. Say: "The pharmacist sees prescriptions as orders to fulfill."

6. **Show the role guard (1 min)**
   While logged in as a patient, type `/dashboard/new-prescription` in the URL. The Unauthorized page shows. Say: "The role guard blocks this even if you know the URL."

7. **Open to questions (1 min)**
   Say: "That covers the main features. I'd love your feedback — what's the first thing you would improve?"

7 The Night Before — Your Checklist

- ✅ Run `npm run dev` — confirm the app starts with no errors
- ✅ Create a doctor test account and log in — confirm dashboard shows
- ✅ Create a patient test account — confirm doctor can connect to patient
- ✅ Create a pharmacy test account — confirm inventory page shows
- ✅ Write one test prescription from the doctor account
- ✅ Add 2–3 medicines to the pharmacy inventory
- ✅ Read Sections 3, 4, and 5 of this guide one more time
- ✅ Say the opening sentence out loud: *"I built MedFind — a prescription management web app..."*
- ✅ Sleep. A rested brain explains things better than a tired one.

The single most important thing to remember

You built this. You made the decisions. You know what the app does. Your mentor is on your side — they want you to succeed.

You don't need to know everything. You need to know your own project.